ITA06 – MACHINE LEARNING

ASSIGNMENT 2 – CO2

Name: M. Pranay

Register No.: 192525382

Q1 – Design a Neural Network Representation for a Classification Problem (6 Marks)

1.1 Input features and numerical representation

Each component is described by three continuous measurements, so the input is a vector $\mathbf{x} = (x_1, x_2, x_3)^T \in \mathbb{R}^3$:

Table 1. Input features and target

Feature	Physical quantity (example unit)	Representation
x_1 Surface roughness	Ra in µm	Real number (z-scored)
x_2 Dimensional deviation	Deviation from nominal size in mm (signed, or absolute value if only the magnitude matters)	Real number (z-scored)
x_3 Vibration level	RMS vibration in mm/s	Real number (z-scored)
Label y	Acceptable = 0, Defective = 1	Binary target; network outputs P(y = 1)

1.2 Preprocessing and why it is necessary

- **Standardisation (z-score)** $x'_i = (x_i - \mu_i)/\sigma_i$, with μ_i, σ_i computed on the training set only. The three features have different units and ranges (µm, mm, mm/s). Without scaling, the feature with the largest numerical range dominates the weighted sum, gradients for the different weights have very different magnitudes (poorly conditioned loss surface, slow or erratic gradient descent) and sigmoid/tanh units saturate early.
- **Fit the scaler on training data only**, then apply the same µ, σ to validation, test and production data; otherwise information from unseen data leaks into training.
- **Data cleaning:** remove or impute missing sensor readings and check outliers (a faulty sensor spike would distort µ and σ).
- **Class imbalance:** defective parts are usually rare. Use a stratified split and consider class weights or a lower decision threshold so the network is not rewarded for predicting "Acceptable" every time.

1.3 Two candidate architectures

Both architectures have **3 input neurons** (one per feature) and **1 output neuron**. A single sigmoid output is sufficient for a binary problem: $\hat{y} = \sigma(z)$ is interpreted as $P(\text{Defective} \mid x)$. Output activation: **sigmoid**, trained with binary cross-entropy. Hidden activation in architecture B: **ReLU** (cheap, does not saturate for positive inputs; tanh would also be acceptable).

Output decision rule: predict Defective (1) if $\hat{y} \geq \tau$, otherwise Acceptable (0). The default is $\boldsymbol{\tau = 0.5}$. Because a missed defect is normally more costly than a false alarm, τ can be lowered (for example to 0.3–0.4) after examining the precision/recall trade-off on validation data.

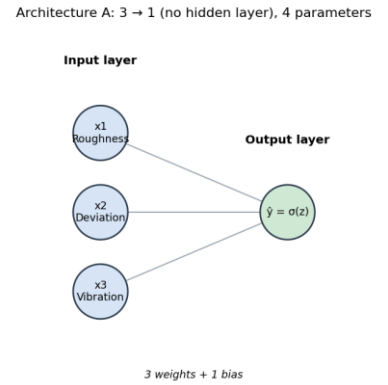


Figure 1. Architecture A – no hidden layer (3 → 1).

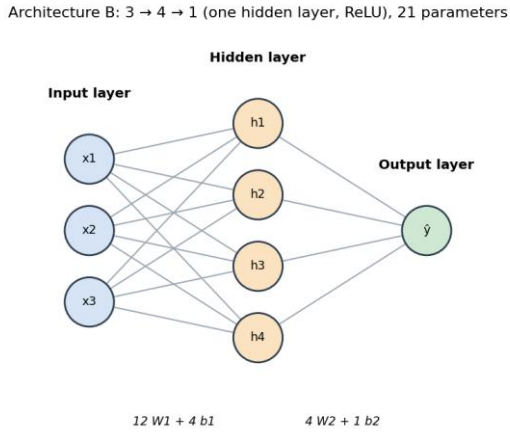


Figure 2. Architecture B – one hidden layer with 4 ReLU neurons (3 → 4 → 1).

1.4 Parameter counts

Table 2. Trainable parameters

Architecture	Weights	Biases	Total trainable
A: 3 → 1	3 × 1 = 3	1	4
B: 3 → 4 → 1	3 × 4 + 4 × 1 = 12 + 4 = 16	4 + 1 = 5	21

1.5 Mathematical computation from input to output

Architecture A (no hidden layer):

$$z = w_1x_1 + w_2x_2 + w_3x_3 + b = w^Tx + b, \quad \hat{y} = \sigma(z) = 1 / (1 + e^{-z})$$

Architecture B (one hidden layer, m = 4 units):

$$z^{(1)} = W^{(1)T}x + b^{(1)} \quad (W^{(1)} \in \mathbb{R}^{3x4}, b^{(1)} \in \mathbb{R}^4), \quad h = \mathrm{ReLU}(z^{(1)}) = \max(0, z^{(1)})$$

$$z^{(2)} = w^{(2)T}h + b^{(2)}, \quad \hat{y} = \sigma(z^{(2)})$$

1.6 Worked forward pass

A component has raw readings **Ra = 1.6 µm, deviation = 0.04 mm, vibration = 3.2 mm/s**. The training set gave µ = (1.2, 0.03, 2.5) and σ = (0.4, 0.02, 1.4). Standardised input:

$$x = ((1.6 - 1.2)/0.4,\ (0.04 - 0.03)/0.02,\ (3.2 - 2.5)/1.4) = (1.0, 0.5, 0.5)$$

Architecture A with assumed small values w = (0.8, 0.6, 0.9), b = −1.0:

$$z = 0.8(1.0) + 0.6(0.5) + 0.9(0.5) - 1.0 = 0.80 + 0.30 + 0.45 - 1.00 = 0.55$$

$$\hat{y} = \sigma(0.55) = 1/(1 + e\text{^}{-}0.55) = 0.634$$

Since 0.634 ≥ 0.5, the component is classified **Defective (1)**.

Architecture B with assumed values (columns of $W^{(1)}$ belong to $h_1 \ldots h_4$):

Table 3. Assumed parameters of architecture B

$W^{(1)}$	h_1	h_2	h_3	h_4	
from x_1	0.5	−0.3	0.2	0.1	
from x_2	0.4	0.6	−0.5	0.3	
from x_3	−0.2	0.5	0.7	−0.4	
$b^{(1)}$	0.1	−0.1	0.0	0.2	
$w^{(2)}$, $b^{(2)}$	0.6	−0.4	0.8	0.5	$b^{(2)} = -0.3$

$$z^{(1)}_1 = 0.5(1.0) + 0.4(0.5) - 0.2(0.5) + 0.1 = 0.70$$
$$z^{(1)}_2 = -0.3(1.0) + 0.6(0.5) + 0.5(0.5) - 0.1 = 0.15$$
$$z^{(1)}_3 = 0.2(1.0) - 0.5(0.5) + 0.7(0.5) + 0.0 = 0.30$$
$$z^{(1)}_4 = 0.1(1.0) + 0.3(0.5) - 0.4(0.5) + 0.2 = 0.25$$

All pre-activations are positive, so ReLU leaves them unchanged: **h = (0.70, 0.15, 0.30, 0.25)**.

$$z^{(2)} = 0.6(0.70) - 0.4(0.15) + 0.8(0.30) + 0.5(0.25) - 0.3 = 0.42 - 0.06 + 0.24 + 0.125 - 0.30 = 0.425$$
$$\hat{y} = \sigma(0.425) = 0.605 \geq 0.5 \rightarrow \text{Defective } (1)$$

1.7 Design inquiry – interaction between measurements

If the inspection team is right that a component is defective **only when certain measurements interact**, the output must depend on combinations of features rather than on their separate effects.

- **Architecture A is additive.** Its log-odds are $w_1x_1 + w_2x_2 + w_3x_3 + b$, so each measurement contributes independently and the decision boundary is a single flat plane. It can represent a monotone "high roughness AND high vibration" rule only as a tilted plane, and it cannot represent rules such as "defective when roughness is high and vibration is low, or the reverse, but not when both are high" (an XOR-type interaction) or a closed "window" of risky values.
- **Architecture B can form interactions.** Each ReLU unit forms a different weighted combination of the three measurements and bends the decision surface; the output neuron combines these bends. A modest hidden layer therefore allows interacting conditions to be represented.

Evidence needed to justify the hidden layer (the hidden layer should be earned, not assumed):

1. Architecture A, properly trained and tuned, **underfits**: training loss/error stays high and does not improve with learning-rate tuning or longer training.
2. Adding explicit interaction terms (x_1x_3, x_2x_3, …) to A, or inspecting residuals/two-feature slices, shows a clearly non-planar pattern.
3. B achieves **better validation** loss, accuracy and (especially) defect recall than A, repeated over several seeds or cross-validation folds, using the same preprocessing and partitions. Gains on training data alone are not sufficient because extra parameters can simply memorise.
4. The gain is large enough to justify the cost (21 vs 4 parameters is trivial in cost, but loss of interpretability and the need for more data are real costs).

Recommendation: use A as the baseline and **B (3 → 4 → 1) as the working design**, because the team's hypothesis is interaction. Keep B only if the validation evidence above supports it; otherwise the simpler, more interpretable A is preferable.

Q2 – Investigate the Learning Behaviour and Limitations of a Perceptron (4 Marks)

2.1 Truth tables

Table 4. Input–output tables for the original requirement (AND) and the changed requirement (XOR)

Packaging Complete (x_1)	Quality Check Passed (x_2)	Dispatch (AND)	Review flag (XOR)
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

2.2 Prediction and geometric reasoning

Prediction: a perceptron **can** learn the dispatch rule (AND) but **cannot** learn the review flag (XOR).

- A perceptron decides with the line $w_1x_1 + w_2x_2 + b = 0$. For AND the single positive point (1,1) can be separated from the three negative points by a line such as $x_1 + x_2 - 1.5 = 0$, so the classes are **linearly separable** and the perceptron convergence theorem guarantees convergence in a finite number of updates.
- For XOR the positive points (0,1), (1,0) lie on one diagonal and the negative points (0,0), (1,1) on the other. No single straight line separates them.

Algebraic proof for XOR. A perceptron would need $b < 0$ (for (0,0)), $w_2 + b \geq 0$, $w_1 + b \geq 0$, and $w_1 + w_2 + b < 0$. Adding the middle two inequalities gives $w_1 + w_2 + 2b \geq 0$, i.e. $w_1 + w_2 + b \geq -b > 0$, which contradicts the last requirement. No weights exist.

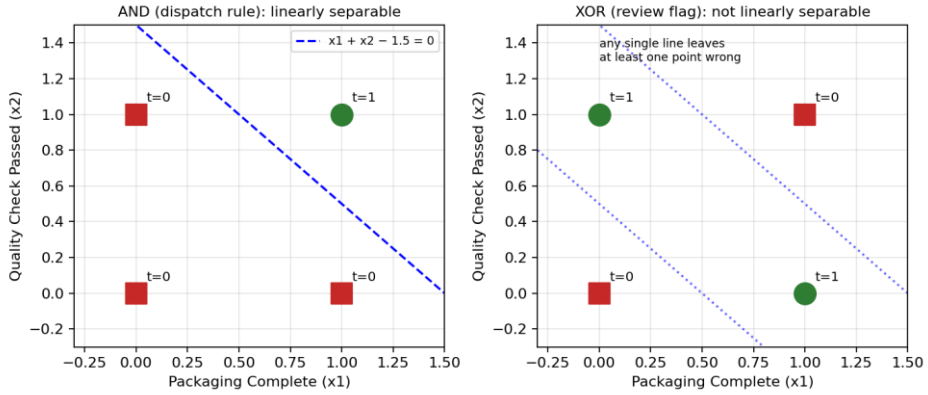


Figure 3. Geometry of the two decision problems: AND is separable by one line, XOR is not.

2.3 Perceptron specification and implementation

Table 5. Perceptron settings

Item	Value
Activation / threshold	Step function: $\hat{y} = 1$ if $z = w \cdot x + b \geq 0$, else 0 (threshold 0)
Initial weights, bias	$w = (0.2, -0.1)$, $b = 0.05$
Learning rate	$\eta = 0.1$ (main run); $\eta = 1.0, 0.1, 0.01$ for the changed requirement
Update rule	$w \leftarrow w + \eta(t - \hat{y})x$, $b \leftarrow b + \eta(t - \hat{y})$
Sample order	(0,0), (0,1), (1,0), (1,1) in every epoch
Maximum epochs	50 for AND, 100 for XOR (stop early if an epoch has 0 errors)

```
import numpy as np
X = np.array([[0,0],[0,1],[1,0],[1,1]])

def perceptron(T, eta, w, b, max_epochs=50):
    w = np.array(w, float)
    errors_per_epoch = []
    for epoch in range(max_epochs):
        errors = 0
        for x, t in zip(X, T):
            y_hat = int(x @ w + b >= 0)          # step activation
            if y_hat != t:                        # misclassified -> update
                w = w + eta * (t - y_hat) * x
                b = b + eta * (t - y_hat)
                errors += 1
        errors_per_epoch.append(errors)
        if errors == 0:                           # converged
            break
    return w, b, errors_per_epoch
```

2.4 Dispatch rule (AND) – training records

Predictions before training (w = (0.2, −0.1), b = 0.05): inputs (0,0), (0,1), (1,0), (1,1) give z = 0.05, −0.05, 0.25, 0.15, hence predictions **1, 0, 1, 1** against targets 0, 0, 0, 1 – two errors.

Table 6. Weight and bias updates during the first epoch (η = 0.1)

Sample	z	ŷ	t	Update?	w after	b after
(0,0)	0.05	1	0	−	(0.20, -0.10)	-0.05
(0,1)	-0.15	0	0	none	(0.20, -0.10)	-0.05
(1,0)	0.15	1	0	−	(0.10, -0.10)	-0.15
(1,1)	-0.15	0	1	+	(0.20, 0.00)	-0.05

Sample (0,0): z = 0.05 gives ŷ = 1 but t = 0, so b ← 0.05 − 0.1 = −0.05 (w is unchanged because x = 0). Sample (1,0): z = 0.15 → ŷ = 1, t = 0, so w_1 ← 0.2 − 0.1 = 0.1 and b ← −0.15. Sample (1,1): z = −0.15 → ŷ = 0, t = 1, so w ← (0.2, 0.0) and b ← −0.05. Misclassifications in epoch 1: **3**.

Table 7. Misclassifications per epoch – AND, η = 0.1

Epoch	1	2	3	4	5	6
Misclassifications	3	2	3	2	1	0

Training converged in **6 epochs** with final **w = (0.20, 0.10), b = -0.25**. Check: (0,0): −0.25 → 0; (0,1): −0.15 → 0; (1,0): −0.05 → 0; (1,1): 0.05 → 1. **Final predictions: 0, 0, 0, 1**, identical to the target column. The learning rate only changes how fast this happens (AND converges in 4, 6 and 23 epochs for η = 1.0, 0.1, 0.01).

2.5 Changed requirement (XOR review flag)

The same perceptron, same initial weights and same stopping rule were trained on the XOR targets with three learning rates.

Table 8. Perceptron on XOR – training record

Rate	Errors in epochs 1–10	Min errors over 100 epochs	Epochs run	Final w, b	Final predictions (target 0,1,1,0)
η = 1.0	3, 3, 4, 4, 4, 4, 4, 4, 4, 4	3	100 (limit)	(-0.80, -0.10), 0.05	1,0,0,0
η = 0.1	3, 2, 2, 3, 4, 4, 4, 4, 4, 4	2	100 (limit)	(-0.10, -0.10), 0.05	1,0,0,0
η = 0.01	3, 3, 3, 3, 3, 2, 2, 2, 2, 2	1	100 (limit)	(-0.01, -0.01), 0.01	1,0,0,0

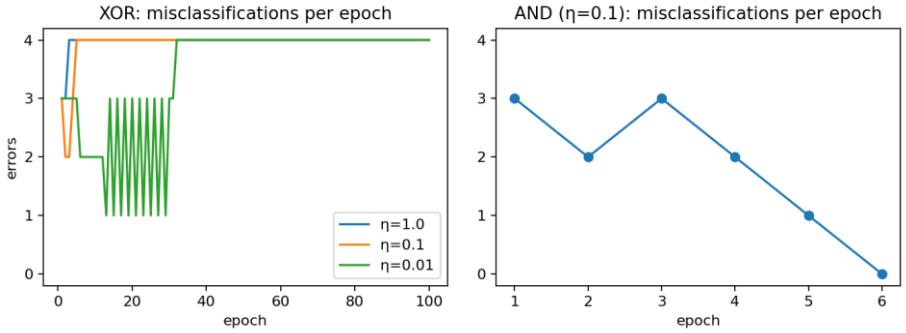


Figure 4. Misclassifications per epoch: AND converges to zero; XOR never does, whatever the learning rate.

In an extra check with 20 random initial (w, b) values and η ∈ {1.0, 0.01}, 200 epochs each, the lowest per-epoch error count reached by any run was **1**; no run ever reached 0 errors.

Does changing the learning rate resolve the difficulty? No. The error count oscillates between 1 and 4 for every η because the weights keep cycling. A different learning rate changes the size of each correction but cannot change the geometry of the model.

Training-configuration problem vs. representation limitation:

- A **configuration problem** (learning rate too small → slow, too large → overshoot, poor initial weights, too few epochs) is fixable by changing those settings. The AND experiment shows this: all rates converge, only the speed differs.
- XOR is a **representation limitation**: the hypothesis class “one linear threshold unit” contains no function that matches the XOR table (proved algebraically in 2.2). Evidence: failure for three learning rates, 20 random initialisations, and 100–200 epochs. The remedy is a different model – a hidden layer with a non-linear activation, developed in Q3.

Q3 – Develop and Verify Backpropagation for a Multilayer Perceptron (6 Marks)

3.1 Network, forward pass and loss

Network: **2 input → 2 hidden (sigmoid) → 1 output (sigmoid)**, 9 parameters ($W^{(1)}$ 2×2, $b^{(1)}$ 2, $w^{(2)}$ 2, $b^{(2)}$ 1). For one sample (x_1, x_2) with target y:

$$z_j^{(1)} = W_{1j}x_1 + W_{2j}x_2 + b_j^{(1)}, \quad h_j = \sigma(z_j^{(1)}), \quad j = 1, 2$$

$$z^{(2)} = w_1^{(2)}h_1 + w_2^{(2)}h_2 + b^{(2)}, \quad \hat{y} = \sigma(z^{(2)})$$

$$L = -[\, y \ln \hat{y} + (1 - y) \ln(1 - \hat{y}) \,] \quad \text{(mean over the N samples of a batch)}$$

3.2 Gradient derivation

Using σ′(z) = σ(z)(1 − σ(z)):

Output layer. $dL/d\hat{y} = -y/\hat{y} + (1 - y)/(1 - \hat{y})$ and $d\hat{y}/dz^{(2)} = \hat{y}(1 - \hat{y})$. Multiplying, the sigmoid derivative cancels the denominators:

$$\delta^{(2)} = \partial L/\partial z^{(2)} = \hat{y} - y$$

$$\partial L/\partial w_j^{(2)} = \delta^{(2)} h_j, \quad \partial L/\partial b^{(2)} = \delta^{(2)}$$

Hidden layer. The loss reaches h_j only through $z^{(2)}$, with $\partial z^{(2)}/\partial h_j = w_j^{(2)}$, and h_j depends on $z_j^{(1)}$ through σ′:

$$\delta_j^{(1)} = \partial L/\partial z_j^{(1)} = \delta^{(2)} \cdot w_j^{(2)} \cdot h_j(1 - h_j)$$

$$\partial L/\partial W_{ij}^{(1)} = \delta_j^{(1)} x_i, \quad \partial L/\partial b_j^{(1)} = \delta_j^{(1)}$$

For a mini-batch, the gradients above are averaged over the N samples.

3.3 Update equations (gradient descent, learning rate η)

$$w_j^{(2)} \leftarrow w_j^{(2)} - \eta\, \delta^{(2)} h_j, \quad b^{(2)} \leftarrow b^{(2)} - \eta\, \delta^{(2)}$$

$$W_{ij}^{(1)} \leftarrow W_{ij}^{(1)} - \eta\, \delta_j^{(1)} x_i, \quad b_j^{(1)} \leftarrow b_j^{(1)} - \eta\, \delta_j^{(1)}$$

3.4 Why the hidden activation must be non-linear

If the hidden units were linear ($h = W^{(1)T}x + b^{(1)}$), the output would be $\sigma(w^{(2)T}(W^{(1)T}x + b^{(1)}) + b^{(2)}) = \sigma(w'^{T}x + b')$ with $w' = W^{(1)}w^{(2)}$ and $b' = w^{(2)T}b^{(1)} + b^{(2)}$. A composition of linear maps is again a linear map, so the whole network collapses to a single-layer model with one straight boundary – exactly the model that failed on XOR in Q2. A non-linear hidden activation (here sigmoid) lets each hidden unit carve its own half-plane and the output unit combine them, e.g. XOR = (x_1 OR x_2) AND NOT (x_1 AND x_2). Non-linearity also makes depth meaningful: the gradient involves σ′ at every layer, which is what backpropagation propagates.

3.5 Manual calculation of one training step

Stated initial parameters ($W^{(1)}$ row i holds the weights from x_i to h_1, h_2): $W^{(1)} = [[0.5, -0.4], [0.3, 0.8]]$, $b^{(1)} = (0.1, -0.2)$, $w^{(2)} = (0.7, -0.6)$, $b^{(2)} = 0.05$. Training example **x = (1, 0), y = 1**, learning rate **η = 0.5**.

Forward pass

$$z_1^{(1)} = 0.5(1) + 0.3(0) + 0.1 = 0.6, \quad z_2^{(1)} = -0.4(1) + 0.8(0) - 0.2 = -0.6$$

$$h_1 = \sigma(0.6) = 0.6457, \quad h_2 = \sigma(-0.6) = 0.3543$$

$$z^{(2)} = 0.7(0.6457) - 0.6(0.3543) + 0.05 = 0.2894, \quad \hat{y} = \sigma(z^{(2)}) = 0.5718$$

$$L = -\ln(\hat{y}) = -\ln(0.5718) = 0.5589$$

Backward pass

$$\delta^{(2)} = \hat{y} - y = 0.5718 - 1 = -0.4282$$

$$\partial L/\partial w_1^{(2)} = \delta^{(2)} h_1 = -0.2764, \quad \partial L/\partial w_2^{(2)} = \delta^{(2)} h_2 = -0.1517, \quad \partial L/\partial b^{(2)} = -0.4282$$

$$\delta_1^{(1)} = \delta^{(2)}\, w_1^{(2)}\, h_1(1 - h_1) = (-0.4282)(0.7)(0.2288) = -0.0686$$

$$\delta_2^{(1)} = \delta^{(2)}\, w_2^{(2)}\, h_2(1 - h_2) = (-0.4282)(-0.6)(0.2288) = 0.0588$$

$$\partial L/\partial W_{11} = \delta_1^{(1)} x_1 = -0.0686, \quad \partial L/\partial W_{12} = \delta_2^{(1)} x_1 = 0.0588, \quad \partial L/\partial W_{21} = \partial L/\partial W_{22} = 0 \quad (x_2 = 0)$$

Updates ($\theta \leftarrow \theta - 0.5 \cdot \text{gradient}$):

Table 9. Parameters before and after one training step

Parameter	Before	After
$W^{(1)}$	[[0.5, −0.4], [0.3, 0.8]]	[[0.5343, -0.4294], [0.3, 0.8]]
$b^{(1)}$	(0.1, −0.2)	(0.1343, -0.2294)
$w^{(2)}$	(0.7, −0.6)	(0.8382, -0.5241)
$b^{(2)}$	0.05	0.2641
Loss on the example	0.5589	0.4236

The loss for this example falls from **0.5589 to 0.4236**, confirming that the step moved the parameters in a descent direction.

3.6 Implementation (NumPy only)

```
import numpy as np
def sigmoid(z): return 1 / (1 + np.exp(-z))

def forward(P, X):
    W1, b1, W2, b2 = P
    H = sigmoid(X @ W1 + b1)            # hidden outputs  (N x 2)
    O = sigmoid(H @ W2 + b2)            # predictions     (N x 1)
    return H, O

def loss(P, X, y):
    _, O = forward(P, X); O = np.clip(O.ravel(), 1e-12, 1 - 1e-12)
    return -np.mean(y*np.log(O) + (1 - y)*np.log(1 - O))

def backward(P, X, y):                  # analytical gradients
    W1, b1, W2, b2 = P
    H, O = forward(P, X); n = len(y)
    d2 = (O - y.reshape(-1, 1)) / n     # dL/dz2
    gW2, gb2 = H.T @ d2, d2.sum(0)
    d1 = (d2 @ W2.T) * H * (1 - H)      # dL/dz1
    gW1, gb1 = X.T @ d1, d1.sum(0)
    return [gW1, gb1, gW2, gb2]

def numerical_grad(P, X, y, eps=1e-5):  # central finite differences
    G = []
    for p in P:
        g = np.zeros_like(p)
        for idx in np.ndindex(p.shape):
            old = p[idx]
            p[idx] = old + eps; Lp = loss(P, X, y)
            p[idx] = old - eps; Lm = loss(P, X, y)
            p[idx] = old; g[idx] = (Lp - Lm) / (2*eps)
        G.append(g)
    return G
```

```
# training: full batch, lr = 1.0, 10 000 epochs
# for _ in range(10000): g = backward(P, X, y); P = [p - 1.0*gi for p, gi in zip(P, g)]
```

3.7 Validation – numerical gradient check

All 9 parameters of a randomly initialised network were checked on the four XOR samples (central differences, $\varepsilon = 10^{-5}$). Relative error = $|g_{num} - g_{ana}| / (|g_{num}| + |g_{ana}|)$.

Table 10. Analytical vs finite-difference gradients

Parameter	Analytical gradient	Numerical gradient	Relative error
W1[0, 0]	0.06253878	0.06253878	1.7e-11
W1[0, 1]	0.00285632	0.00285632	3.9e-10
W1[1, 0]	0.10675111	0.10675111	3.3e-11
W1[1, 1]	0.01077360	0.01077360	3.4e-10
b1[0]	0.15505436	0.15505436	9.5e-12
b1[1]	0.01164189	0.01164189	2.8e-10
W2[0, 0]	-0.27109099	-0.27109099	1.7e-11
W2[1, 0]	-0.06510360	-0.06510360	3.2e-11
b2[0]	-0.39699601	-0.39699601	1.8e-11

The maximum relative error is **3.9e-10**, far below the usual 10^{-6} acceptance level, so the backpropagation code is correct.

3.8 Training on all four input combinations

Settings: full-batch gradient descent, $\eta = 1.0$, 10 000 epochs, weights drawn from N(0, 1), biases initially 0. Ten random seeds were run to see whether success depends on initialisation.

Table 11. Final outputs for ten initialisations

Seed	Final loss	Final outputs for (0,0), (0,1), (1,0), (1,1)	Accuracy	Outcome
0	0.0010	[0.001, 0.999, 0.999, 0.001]	1.00	learned XOR
1	0.0010	[0.001, 0.999, 0.999, 0.001]	1.00	learned XOR
2	0.0010	[0.001, 0.999, 0.999, 0.001]	1.00	learned XOR
3	0.0010	[0.001, 0.999, 0.999, 0.001]	1.00	learned XOR
4	0.0010	[0.001, 0.999, 0.999, 0.001]	1.00	learned XOR
5	0.0011	[0.001, 0.999, 0.998, 0.001]	1.00	learned XOR
6	0.3472	[0.001, 0.500, 0.999, 0.500]	0.50	stuck
7	0.0010	[0.001, 0.999, 0.999, 0.001]	1.00	learned XOR
8	0.0010	[0.001, 0.999, 0.999, 0.001]	1.00	learned XOR
9	0.0010	[0.001, 0.999, 0.999, 0.001]	1.00	learned XOR

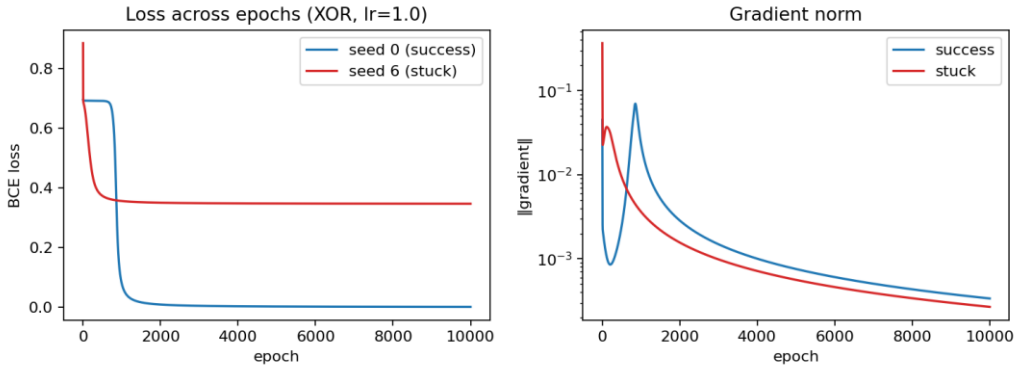


Figure 5. Loss and gradient norm across epochs: seed 0 (successful) and seed 6 (stuck).

Successful runs (9 of 10). The loss starts near 0.69 (= ln 2, outputs ≈ 0.5), stays on a plateau for several hundred epochs while the hidden units differentiate, then falls quickly to about 10^{-3}. For seed 0 the final outputs are **(0.0010, 0.9991, 0.9987, 0.0008)**, i.e. 0.001, 0.999, 0.999, 0.001 for the targets 0, 1, 1, 0 – correct XOR with confident predictions.

The failure (seed 6) investigated with evidence, not just final accuracy:

- Final outputs were (0.001, **0.500**, 0.999, **0.500**) with loss 0.347. This is exactly two samples learned perfectly plus two stuck at $\hat{y} = 0.5$: $(2 \times 0.693)/4 = 0.347$. Accuracy 50 %.
- The loss curve flattens at 0.347 and the gradient norm falls to $\approx 3 \times 10^{-4}$: the optimiser is not diverging or oscillating, it has **converged to a poor stationary point**.
- Inspecting the hidden layer shows why: for inputs (0,1) and (1,1) the hidden activations are both ≈ (0.0, 1.0). The hidden units have collapsed onto the same representation for these two inputs, so the output neuron cannot give them different answers. The large initial weights (e.g. −2.55) pushed a hidden sigmoid into saturation where $\sigma' \approx 0$, so almost no gradient reaches those weights to correct the collapse (vanishing-gradient effect).
- Changing the learning rate did **not** repair it: for the same initial weights $\eta = 0.5$, 1, 2 and 4 all ended at accuracy 0.5 and loss ≈ 0.347. This points to an initialisation/optimisation-landscape problem (saturated sigmoids, local plateau), not a step-size problem and not a lack of representational power – nine other initialisations solve XOR with the same architecture.

Remedy: use a smaller initial scale (e.g. $N(0, 0.5^2)$), several random restarts, or non-saturating hidden units (tanh/ReLU) – the same lesson reappears in Q6.

Q4 – Investigate Whether a Deeper Network Is a Better Design (6 Marks)

4.1 Data and experimental design

Data: two interleaving crescents, 1000 points, noise 0.25, split 600/200/200. The two classes overlap slightly, so a perfect score is not possible; the achievable accuracy is limited by label noise at roughly 94–96 %.

Table 12. Architectures compared

Model	Hidden layers	Parameters	Width justification
Shallow	1 × 32	129 (2·32+32 + 32+1)	32 ReLU units are enough pieces to bend a boundary around two crescents, with a safety margin.
Deep	3 × 16	609 (48 + 272 + 272 + 17)	Moderate widths so depth, not width, supplies the capacity; 16 units per layer are enough to compose simple features.
Wide control	1 × 152	609 (4·152+1)	Extra control: a one-hidden-layer network with exactly the deep network's parameter count, to separate the effect of depth from the effect of size.

Common settings: ReLU hidden units, sigmoid output, binary cross-entropy, He initialisation, Adam (lr 0.01), batch size 32, **150 epochs for every model**, five seeds (0–4). The epoch with the lowest validation loss is chosen for each run (architecture/epoch selection on validation data only); the test set is evaluated once afterwards with that checkpoint.

4.2 Predictions made before training

- **Representation:** both can represent the crescent boundary (a 2-D boundary needs only a handful of piecewise-linear pieces). The deep network composes simple features layer by layer and is more parameter-efficient for very intricate boundaries, but this problem is simple.
- **Training behaviour:** the deep network has more layers to propagate gradients through, so I expected somewhat slower and less stable optimisation and larger seed-to-seed variation.
- **Generalisation:** with 600 training points and label noise, I expected similar test accuracy; neither network should clearly beat the other.
- **Computational effort:** the deep network needs about twice the time per epoch (more layers, more matrix products).

4.3 Results (mean ± SD over 5 seeds, at the best-validation epoch)

Table 13. Shallow vs deep vs parameter-matched wide network

Model	Params	Train loss	Val loss	Train acc	Val acc	Test acc	Time (s)
Shallow (2-32-1)	129	0.126 ± 0.004	0.136 ± 0.004	0.944 ± 0.001	0.960 ± 0.007	0.939 ± 0.014	0.24 ± 0.00
Deep (2-16-16-16-1)	609	0.117 ± 0.005	0.120 ± 0.003	0.948 ± 0.003	0.958 ± 0.008	0.934 ± 0.011	0.48 ± 0.03
Wide control (2-152-1)	609	0.124 ± 0.005	0.133 ± 0.001	0.945 ± 0.002	0.966 ± 0.004	0.940 ± 0.010	0.36 ± 0.01

Table 14. Epoch of best validation loss and test loss

Model	Best-val epoch	Test loss	Test accuracy
Shallow (2-32-1)	93 ± 45	0.134 ± 0.004	0.939 ± 0.014
Deep (2-16-16-16-1)	61 ± 19	0.140 ± 0.006	0.934 ± 0.011
Wide control (2-152-1)	106 ± 42	0.138 ± 0.005	0.940 ± 0.010

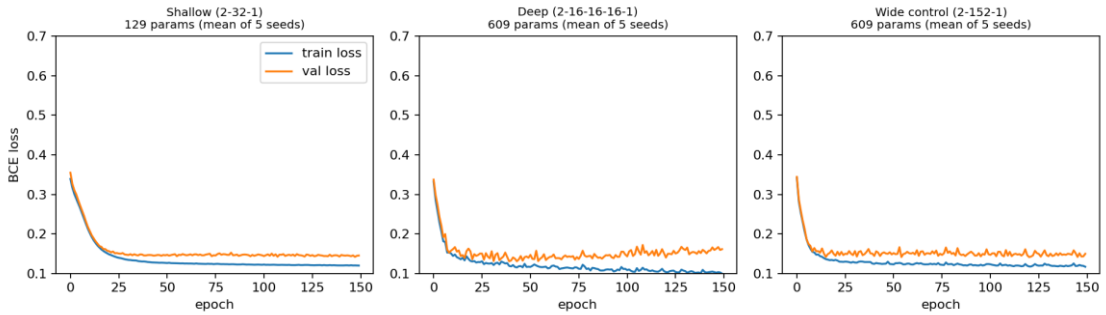


Figure 6. Training and validation loss (mean of 5 seeds) for the three architectures.

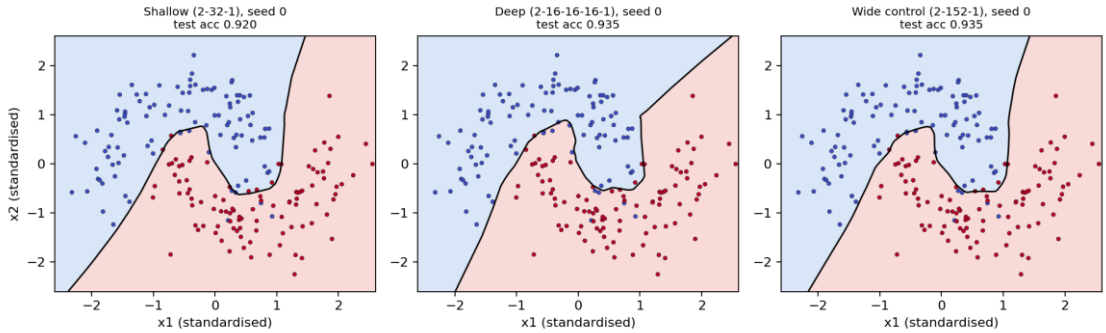


Figure 7. Decision boundaries (seed 0) with the test points overlaid.

4.4 Comparison with the predictions

- **Representation – confirmed.** All three boundaries follow the crescents closely; the only visible differences are small wiggles in regions where the classes overlap.
- **Training behaviour – partly contradicted.** The deep network reached its best validation loss earlier (epoch ≈ 61 vs 93 for the shallow model) and its seed variation was not larger. It was not slower to optimise with Adam on this small problem.
- **Generalisation – confirmed.** Test accuracy is 0.934 (deep), 0.939 (shallow) and 0.940 (wide control), all within one standard deviation (≈ 0.011–0.014) of each other.
- **Computational effort – confirmed.** Deep training took ≈ 0.48 s versus 0.24 s (shallow) – about twice as long – with 4.7× as many parameters.

4.5 Design inquiry and recommendation

Recommendation: the one-hidden-layer network (2-32-1, 129 parameters). It matches the deeper network on validation and test accuracy, uses 4.7× fewer parameters, trains about twice as fast and is easiest to analyse.

Can the improvement be attributed to depth alone? The only measurable "improvement" of the deep network is a lower validation loss (0.120 vs 0.136, about 12 %), not higher accuracy. Because the deep network also has 4.7× more parameters, size is a competing explanation. The parameter-matched wide control separates the two: wide (609 parameters) reaches 0.133, so extra parameters

explain only about 0.003 of the 0.016 improvement, while the remaining ≈ 0.013 appears linked to depth. However, (i) the advantage does not carry over to the test set (test loss 0.140 deep vs 0.134 shallow vs 0.138 wide) and does not appear in accuracy, (ii) only five seeds and one data split were used, and (iii) the deep network needed twice the compute. The evidence therefore supports “depth gives a small validation-loss benefit on this split”, not “depth makes the model better”. For a 2-D problem with this much label noise, depth is not justified.

Q5 – Diagnose Overfitting and Evaluate Regularization (7 Marks)

5.1 Limited-data experiment

Training data: 100 observations (50 per class) drawn at random from the Q4 training partition. **Validation (200) and test (200)** are the same as in Q4. Standardisation uses the mean/SD of these 100 points.

Model: deliberately large – 2-256-256-1 ReLU network with **66,817 parameters** (≈ 668 parameters per training sample). **Common settings:** Adam (lr 0.005), batch size 20, up to 300 epochs, five seeds.

Table 15. Settings of the compared configurations

Intervention	Exact setting
Baseline	No regularization; train all 300 epochs, use the final model
L2 regularization	$\lambda = 1\times10^{-3}$: loss = BCE + $(\lambda/2)\cdot\Sigma\|W\|^2$ (weights, not biases); all 300 epochs, final model
Dropout	Rate p = 0.3 on both hidden layers (inverted dropout, off at evaluation); all 300 epochs, final model
Early stopping	Monitor validation loss, patience 20 epochs, minimum improvement 10^{-4}, restore the weights of the best epoch; maximum 300 epochs

5.2 Prediction before training

With 668 parameters per example and no constraint, the network should be able to memorise the 100 points: **training accuracy ≈ 100 % and training loss near 0, but validation accuracy clearly lower (≈ 88–92 %)** and validation loss that first falls and then rises as the network becomes over-confident on training specifics. The train–validation gap should be large.

5.3 Results

Table 16. Final model of each configuration (mean of 5 seeds)

Configuration	Train loss	Val loss	Train acc	Val acc	Acc gap (train – val)	Weight norm ‖W‖
Baseline	0.038	0.755	0.994	0.923	0.071	52.6
L2 (λ=1e-3)	0.090	0.280	0.970	0.932	0.038	10.7
Dropout (p=0.3)	0.071	0.596	0.970	0.930	0.040	50.3
Early stopping (patience 20)	0.135	0.221	0.934	0.933	0.001	34.2

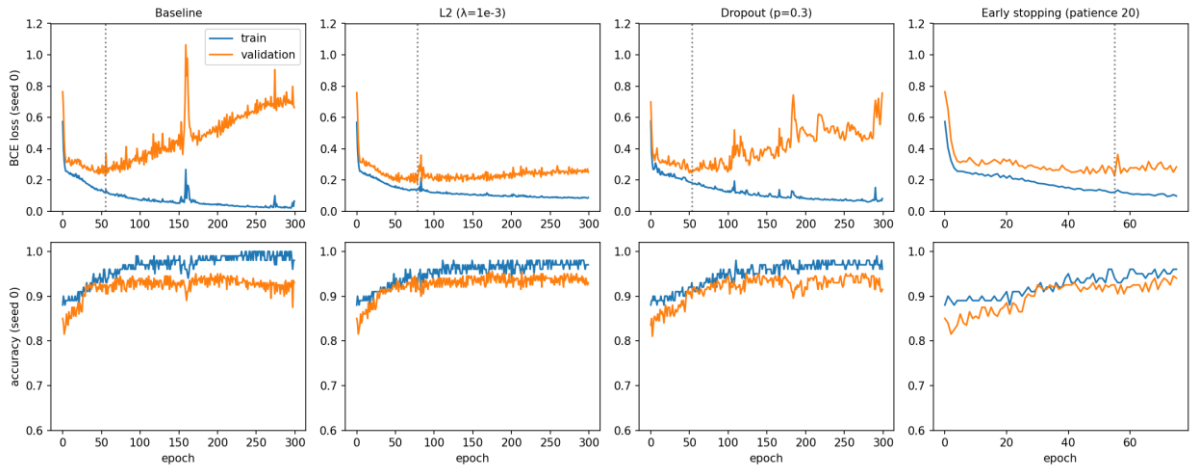


Figure 8. Loss (top) and accuracy (bottom) curves, seed 0. Dotted line: epoch of lowest validation loss.

Regularization-strength sweep (validation only, 5 seeds)

Table 17. Effect of the regularization strength on validation behaviour

Setting	Train acc	Val acc	Val loss	‖W‖
L2 λ = 0.0001	0.986	0.928	0.552	25.9
L2 λ = 0.001	0.970	0.932	0.280	10.7
L2 λ = 0.01	0.910	0.921	0.219	4.4
L2 λ = 0.1	0.890	0.835	0.348	2.0
Dropout p = 0.1	0.984	0.936	0.563	52.3
Dropout p = 0.3	0.970	0.930	0.596	50.3
Dropout p = 0.5	0.966	0.932	0.465	46.8
Dropout p = 0.7	0.926	0.923	0.286	43.5

5.4 Analysis

1. Does the baseline overfit? Yes. Train accuracy 99.4 % vs validation 92.3 % (gap 7.1 points); training loss 0.038 vs validation loss **0.755** (a 20× difference). The validation loss was lowest (≈ 0.22) around epoch 48 and then rose to 0.755 (3.4× worse) while training loss kept falling and the weight norm grew to 52.6. The prediction was confirmed, except that validation accuracy (92 %) stayed higher than expected – accuracy hides the over-confidence that the loss reveals.

2. Most useful improvement: early stopping. It gave the lowest validation loss (0.221) and the highest validation accuracy (0.933), eliminated the accuracy gap (train 0.934 vs val 0.933) and used about 4× less computation (stopped after ≈ 67 epochs instead of 300). L2 (val loss 0.280) was second; dropout improved accuracy (0.930) but left the validation loss high (0.596) – its predictions remain over-confident.

3. Does any setting introduce underfitting? Yes, at excessive strength. L2 with λ = 0.1 gives train accuracy 0.89 and validation accuracy only 0.835 – both low, so the model is too constrained. Dropout with p = 0.7 raises training loss to 0.16 (train accuracy 0.926). The chosen settings ($\lambda = 10^{-3}$, p = 0.3, patience 20) do not underfit: training accuracy remains above 0.93 and equals or exceeds validation accuracy. ($\lambda = 10^{-2}$ gives validation loss 0.219 but validation accuracy falls to 0.921 and train accuracy to 0.91, so it is already at the edge.)

4. How each intervention changes learning behaviour.

- **L2** adds a penalty gradient λW that shrinks weights continuously (norm 52.6 → 10.7). The network fits the data more slowly, the boundary becomes smoother and the validation loss curve flattens instead of climbing.
- **Dropout** injects noise: each update trains a different thinned sub-network, so curves are noisier and the training loss falls more slowly. The weight norm stays large (50.3), and the validation loss still rises late in training, so by itself it was the weakest at calibration.
- **Early stopping** does not change the loss function or the path of the optimiser; it simply stops at the epoch where validation loss is minimal, limiting the effective capacity the network is allowed to use.

5. Does higher training accuracy imply a better model? No. The baseline has the highest training accuracy (0.994) but the worst validation loss (0.755) and lowest validation accuracy (0.923); early stopping has the lowest training accuracy of the four (0.934) and the best validation results. Training accuracy measures memorisation; only held-out data measure generalisation.

5.5 Selection and final test result

Selected configuration (chosen on validation evidence only): early stopping – lowest validation loss (0.221), highest validation accuracy (0.933), no train–validation gap, smallest compute. Only then was the test set evaluated.

Table 18. Test performance after selection (mean ± SD, 5 seeds)

Model	Test accuracy	Test loss
Early stopping (selected)	0.934 ± 0.010	0.200 ± 0.010
Baseline (for reference)	0.921 ± 0.009	0.567 ± 0.095

The test loss drops from 0.567 to 0.200 (≈ 65 % lower), confirming a real generalisation improvement; the accuracy difference (≈ 1.3 points) is about one standard deviation and should be treated as small.

Q6 – Engineer a Hyperparameter and Optimization Strategy (8 Marks)

6.1 Search design (before experimentation)

Model and protocol: 2-w-w-1 ReLU network (two hidden layers of width w), Q4 partitions, standardisation fitted on the training data only, maximum **100 epochs**, early stopping with **patience 15** on validation loss (min. improvement 10^{-4}, best weights restored), seed 0 for every run, batch cross-entropy loss. Momentum coefficient $\beta = 0.9$.

Validation measure for selection: the lowest validation binary cross-entropy attained during the run. **Tie rule (fixed beforehand):** configurations within 0.005 of the best validation loss are treated as equivalent and the one with the shortest training time is chosen. (Validation accuracy and loss-instability are recorded as supporting information.)

Allocation of the 12 runs – a controlled one-factor-at-a-time design around a base configuration R1 (Adam, lr 0.01, batch 32, width 32). Each factor is varied while the others are held at the base, so the difference between two runs isolates one variable. Random or grid search would spend most of the 12 runs on interactions that cannot be resolved with one seed per run.

Table 19. The 12 candidate configurations

Run	Optimizer	lr	Batch	Width	Comparison	What it isolates
R1	Adam	0.01	32	32	Base	Reference
R2	Adam	0.0001	32	32	R2 vs R1	Learning rate too small
R3	Adam	0.001	32	32	R3 vs R1	Smaller learning rate
R4	Adam	0.1	32	32	R4 vs R1	Learning rate too large
R5	Adam	0.01	8	32	R5 vs R1	Small batch
R6	Adam	0.01	128	32	R6 vs R1	Large batch
R7	Adam	0.01	32	8	R7 vs R1	Narrow network
R8	Adam	0.01	32	128	R8 vs R1	Wide network
R9	SGD	0.05	32	32	Reference for SGD family	Plain SGD
R10	Momentum	0.05	32	32	R10 vs R9	Effect of momentum at the same lr
R11	SGD	0.5	32	32	R11 vs R9	Learning rate ×10 for SGD
R12	Momentum	0.005	32	32	R12 vs R9	Momentum with equal effective step (lr/(1−β) ≈ 0.05)

Budget split: 1 base + 3 learning-rate runs + 2 batch-size runs + 2 width runs + 4 optimizer runs. The learning rate gets the most runs because it is normally the most influential hyperparameter and appears in both the Adam and SGD families.

Predictions

- **Learning rate too small (R2, lr = 10^{-4}):** very slow progress; after 100 epochs the loss will still be falling and the validation loss will be clearly worse than the base run (under-trained, not over-fitted).
- **Learning rate too large (R4, lr = 0.1 for Adam; R11, lr = 0.5 for SGD):** noisy, oscillating validation loss with large spikes, possible divergence or a worse minimum.
- **Optimizers:** Adam should converge fastest and be least sensitive to lr; plain SGD should be the slowest; momentum should accelerate SGD (I expect part of that gain to be just a larger effective step size).

- **Batch size and width:** I expect small effects: smaller batches are noisier and slower per epoch; width beyond ≈ 8–32 should add little for a 2-D problem.

6.2 Results

Table 20. Configuration–result table (one run per configuration)

Run	Optim.	lr	Batch	Width	Params	Best val loss	Best val acc	Best epoch	Time (s)	Instability (val-loss jumps >10 %)
R1	Adam	0.01	32	32	1185	0.1292	0.960	42	0.34	7
R2	Adam	0.0001	32	32	1185	0.3022	0.850	100	0.41	0
R3	Adam	0.001	32	32	1185	0.1363	0.945	95	0.44	0
R4	Adam	0.1	32	32	1185	0.1265	0.960	10	0.11	12
R5	Adam	0.01	8	32	1185	0.1240	0.960	18	0.46	5
R6	Adam	0.01	128	32	1185	0.1284	0.945	30	0.08	1
R7	Adam	0.01	32	8	105	0.1258	0.955	36	0.18	6
R8	Adam	0.01	32	128	17025	0.1291	0.965	13	0.34	3
R9	SGD	0.05	32	32	1185	0.1491	0.955	98	0.28	0
R10	Mom.	0.05	32	32	1185	0.1238	0.950	35	0.16	10
R11	SGD	0.5	32	32	1185	0.1266	0.960	30	0.10	10
R12	Mom.	0.005	32	32	1185	0.1518	0.950	99	0.36	0

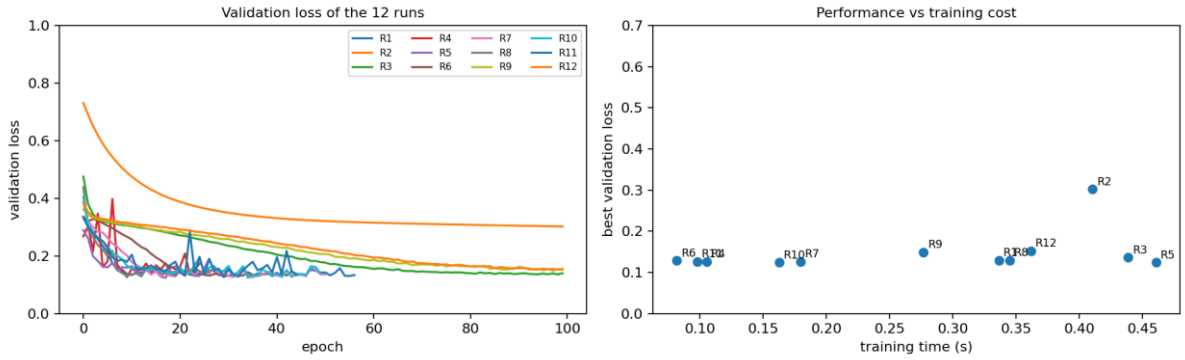


Figure 9. Validation loss of the 12 runs (left) and performance versus training cost (right).

6.3 Optimisation analysis

- **Learning rate too small (R2):** best validation loss 0.302 (vs 0.129 for R1), still improving at epoch 100 (best epoch = 100) – the prediction is confirmed: it is unsuccessful learning because the budget ends before convergence. R3 (10^{-3}) needed 95 epochs to reach 0.136.
- **Learning rate too large (R4, R11):** both showed the most unstable validation curves (12 and 10 loss jumps >10 %) and stopped early. They nevertheless reached good minima (0.127, 0.127). The prediction of failure was **too pessimistic** for this small, well-conditioned problem – Adam's per-parameter normalisation limits the step size, and a 2-D crescent problem is forgiving. Instability still signals a risk that would be larger on harder problems.
- **Optimizer:** SGD (lr 0.05, R9) had not converged by epoch 98 (0.149). Momentum at the same lr (R10) reached 0.124 in 35 epochs. But momentum at a 10× smaller lr (R12), which has the same effective step, behaved like SGD (0.152 at epoch 99), and plain SGD with a 10× larger lr (R11) also reached 0.127. So most of momentum's benefit here is an effective-step-size effect; Adam (R1, 0.129 at epoch 42) removed the need to tune it.

- **Batch size:** 8 → 0.124, 32 → 0.129, 128 → 0.128. The differences are tiny but the cost differs: batch 128 took 0.08 s vs 0.46 s for batch 8 (updates per epoch are 16× fewer).
- **Width:** w = 8 (105 parameters) 0.126, w = 32 (1,185) 0.129, w = 128 (17,025) 0.129 – the problem needs very little capacity.

6.4 Selection inquiry

Best validation loss: R10 = 0.1238. Runs within 0.005 of that value: R4, R5, R6, R7, R10, R11. Applying the pre-declared tie rule (shortest training time), the final choice is **R6: Adam, lr = 0.01, batch size 128, two hidden layers of 32 units** (validation loss 0.1284, validation accuracy 0.945, 0.08 s).

Test performance (evaluated once, after selection; configuration not revised): test accuracy **0.940**, test loss **0.141**. As a robustness note only (not used to change anything), five seeds of the same configuration gave test accuracies of 0.940, 0.935, 0.945, 0.935, 0.945 (mean 0.940).

Factors with the strongest observed influence: (1) learning rate / effective step size – the only factor that moved the validation loss by a large margin (0.302 vs 0.129 for lr 10^{-4} vs 10^{-2}, and the SGD/momentum differences, which were largely step-size differences); (2) optimizer, mostly through its effect on the effective step and on epochs-to-converge; (3) batch size and width had almost no effect on the validation loss (≤ 0.005) but changed cost.

Performance-versus-cost trade-off: six configurations are statistically indistinguishable in validation loss (0.124–0.129) but their cost differs by a factor of ≈ 6 (0.08–0.46 s). The larger batch gives similar quality at the lowest cost, which is why it was selected; the narrow network (R7, 105 parameters) is an equally acceptable cheaper-to-deploy alternative.

Conclusions limited by the small budget: one seed per run (seed-to-seed SD in Q4 was ≈ 0.003–0.004, as large as most differences among the top six); one-factor-at-a-time cannot reveal interactions (e.g. a larger batch may need a larger lr); coarse grids (three learning rates, two batch sizes, two widths); fixed momentum β, no learning-rate schedule, no weight decay; wall-clock times of fractions of a second are noisy. The result is “a good configuration”, not “the best”.

Why the test set must stay outside tuning: every decision made with a data set (which lr, which optimizer, when to stop) fits that data set a little. If the test set were used to choose, its score would be optimistically biased and could no longer estimate performance on new data. Keeping it for one final evaluation preserves an unbiased estimate; revising the configuration after seeing the test result would turn the test set into a second validation set.

Q7 – Diagnose, Redesign and Validate a Neural Network Solution (13 Marks)

Selected solution: the unregularised large network of Q5 (2-256-256-1, 66,817 parameters) trained on 100 labelled observations – the solution that showed the most significant limitation (severe overfitting).

A. Diagnose the learning or computational bottleneck

Table 21. Diagnostic evidence

Evidence from Q5 (baseline, mean of 5 seeds)	Value	Interpretation
Training accuracy / validation accuracy	0.994 / 0.923	Gap of 7.1 points
Training loss / validation loss	0.038 / 0.755	Validation loss 20× larger; over-confident predictions
Epoch of lowest validation loss	≈ 48 of 300	Validation loss rose 3.4× afterwards while training loss kept falling
Weight norm ‖W‖	52.6 (L2 run: 10.7)	Large weights → highly flexible, sharp boundaries
Parameters per training sample	≈ 668	Capacity far exceeds information in 100 labels
Training time	≈ 2–3 s for 300 epochs (timing varies between runs)	≈ 250 of the 300 epochs came after the validation optimum

Diagnosis: the bottleneck is **limited data combined with excessive model capacity** (overfitting), plus wasted computation. It is **not** a representation problem (the network easily represents the boundary), nor a gradient or learning-rate problem (training loss decreased smoothly and reached ≈ 0.04) – the optimiser worked too well on too little data.

B. Define the design objective

Table 22. Measurable redesign objective

Measure	Baseline (A)	Target
Generalisation gap, train acc − val acc	0.071	≤ 0.03
Validation loss	0.755	≤ 0.25
Validation accuracy	0.923	≥ 0.913 (acceptable trade-off: lose at most 1 point)
Parameters	66,817	≤ 1,000 (≥ 60× smaller)
Training time	≈ 3 s	≤ 1 s (≥ 3× faster)

Acceptable trade-offs: a small loss of validation accuracy (≤ 1 point) and slightly higher training loss are acceptable in exchange for a model that generalises with a small gap and costs far less. Larger networks or a different optimiser alone are not considered a valid redesign.

C. Design the revised learning strategy

What changes and why (each change tied to the diagnosis):

Table 23. Redesign components

Change	Property of the problem it addresses	Cost / new limitation
1. Reduce capacity: 2-16-16-1 (337 parameters)	Capacity ≫ information (668 parameters per sample). A 2-D boundary needs few units (Q4, Q6: width 8 already works).	Too small a network could underfit; narrower networks are more seed-sensitive.
2. Keep early stopping (patience 20, restore best weights)	Validation loss minimum occurs early; later epochs only add over-confidence.	Needs a validation set (200 points here); patience is a tunable setting.
3. Add mild L2 ($\lambda = 10^{-3}$)	Keeps weights small to give a smoother boundary and calibrated outputs.	One more hyperparameter; slightly more compute per update.

Revised architecture / procedure: 2-16-16-1 ReLU network, sigmoid output, loss = BCE + $(\lambda/2)\Sigma\|W\|^2$, Adam (lr 0.005, batch 20), max 300 epochs, stop when validation loss has not improved for 20 epochs and restore the best weights. Memory: 337 parameters (≈ 2.6 KB in float64) instead of 66,817 (≈ 0.5 MB); extra computation per epoch is one validation forward pass (200 samples), repaid by shorter training.

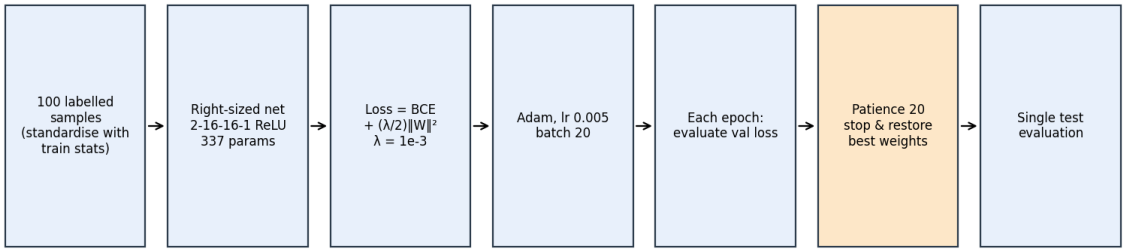


Figure 10. Training workflow of the redesigned solution.

Pseudocode

```
standardise train/val/test with mean and std of the 100 training points
model <- MLP([2,16,16,1], ReLU hidden, sigmoid output, He initialisation)
best_val <- infinity ; wait <- 0
for epoch in 1 .. 300:
    for each mini-batch (size 20) of shuffled training data:
        grads <- backprop( BCE(batch) )  +  lambda * W          # L2 term, lambda = 1e-3
        Adam update (lr = 0.005)
    val_loss <- BCE(model, validation set)
    if val_loss < best_val - 1e-4:  best_val <- val_loss ; save weights ; wait <- 0
    else:                           wait <- wait + 1
    if wait >= 20: break
restore saved weights
evaluate once on the test set
```

D. Prediction before implementing (recorded before running the experiment)

5. The accuracy gap falls from ≈ 7 points to **≤ 3 points**.
6. Validation loss falls from ≈ 0.75 to **≤ 0.25**.
7. Validation accuracy stays at about the baseline value (≥ 0.92); no large gain because label noise sets a ceiling at ≈ 93–94 %.
8. Parameters fall ≈ 200× and training time by **≥ 5×**.
9. Most of the gap reduction comes from capacity reduction plus early stopping; the L2 term adds only a marginal change.

Primary measure expected to improve: generalisation gap and validation loss.

E. Implementation and validation

Same 100 training points, same validation and test sets, same optimizer settings, five seeds. Variants A–E form an ablation so each component can be judged separately.

Table 24. Redesign vs original (mean ± SD over 5 seeds, validation data)

Variant	Params	Train acc	Val acc	Acc gap	Val loss	Time (s)
A Large, no regularization (Q5 baseline)	66817	0.994	0.923 ± 0.007	0.071 ± 0.011	0.755 ± 0.100	3.18 ± 0.17
B Large + early stopping (best Q5 fix)	66817	0.934	0.933 ± 0.005	0.001 ± 0.009	0.221 ± 0.011	0.80 ± 0.12
C Small 2-16-16-1, no regularization	337	0.972	0.936 ± 0.007	0.036 ± 0.009	0.322 ± 0.051	0.29 ± 0.07
D Small + early stopping	337	0.926	0.919 ± 0.032	0.007 ± 0.018	0.214 ± 0.045	0.09 ± 0.04
E Redesign: small + L2 (1e-3) + early stopping	337	0.918	0.918 ± 0.032	0.000 ± 0.018	0.214 ± 0.046	0.11 ± 0.05

Table 25. Epochs run and final test accuracy

Variant	Epochs run	Test accuracy (after selection)
A Large, no regularization (Q5 baseline)	300	0.921 ± 0.009
B Large + early stopping (best Q5 fix)	68	0.934 ± 0.010
E Redesign: small + L2 (1e-3) + early stopping	121	0.927 ± 0.026

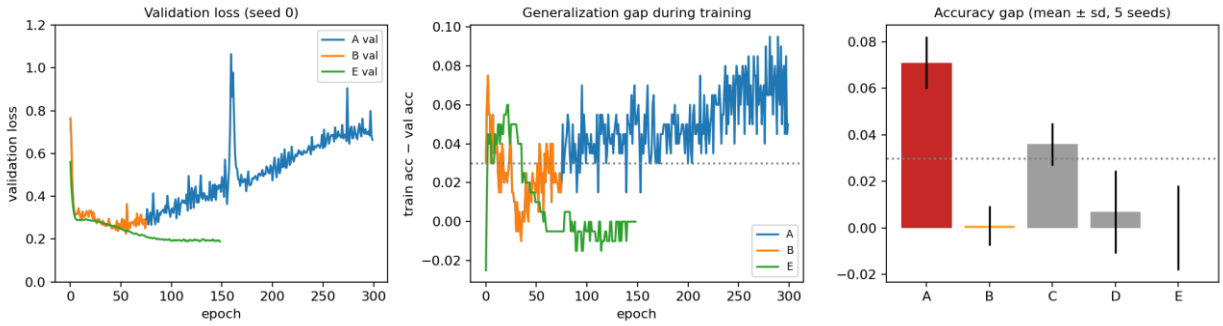


Figure 11. Validation loss, generalisation gap during training, and final accuracy gap for A, B and E.

F. Justify the final design

Did the redesign meet its objective?

Table 26. Objective versus outcome

Target	Result (E)	Predicted?	Met?
Gap ≤ 0.03	0.000 ± 0.018	Yes	Yes
Validation loss ≤ 0.25	0.214 ± 0.046	Yes	Yes
Validation accuracy ≥ 0.913 (objective) / ≥ 0.92 (prediction)	0.918 ± 0.032	Partly	Objective yes; prediction missed by 0.002
Parameters ≤ 1,000	337	Yes	Yes (198× fewer)
Time ≤ 1 s, ≥ 3× faster (prediction: ≥ 5×)	0.11 s	Yes	Yes (28× faster than A, 7× faster than B)

- **Prediction check.** Gap, loss, size and time predictions were confirmed. The accuracy prediction (≥ 0.92) was narrowly missed (0.918); the redesign is 1.5 points below early stopping on the large network (B, 0.933).
- **Ablation:** shrinking the network alone (C) already halved the accuracy gap (0.071 → 0.036) and gave the highest validation accuracy (0.936), but left a high validation loss (0.322). Adding early stopping (D) reduced the loss to 0.214. Adding L2 on top (E) changed almost nothing (loss 0.214 vs 0.214, gap 0.007 vs 0.000) – as predicted, **L2 is marginal here** and could be dropped.
- **New costs and observed side-effects:** the small network is more seed-sensitive (validation-accuracy SD 0.032 vs 0.005 for B) and slightly under-fits (train accuracy 0.918 vs 0.972 for C); early stopping on a noisy validation curve can stop at a poor point for some seeds.
- **When the original (large + early stopping, B) remains preferable:** when accuracy consistency matters more than model size (B has the highest and most stable validation accuracy, 0.933 ± 0.005, test 0.934); when more labelled data become available (a 66,817-parameter network benefits from data and the gap shrinks); or when the true boundary becomes much more complex than two crescents.

Final design decision: adopt **design E (2-16-16-1 with early stopping and mild L2)** when model size, training cost or latency matter – it meets the diagnosed objective with 198× fewer parameters and 28× less training time, at a validation-accuracy cost (0.5 points relative to the baseline, within one standard deviation) that was declared acceptable beforehand. Use **B** instead when the priority is the most reliable accuracy. In both cases, the pure unregularised baseline A should not be used: its main weakness – memorisation of 100 points – was diagnosed and shown to be removed by the redesign. Test accuracies of A, B and E (0.921, 0.934, 0.927) are within seed noise, so the choice rests on validation evidence and cost, not on test differences.

Appendix – Core NumPy library used for Q4–Q7

The following module implements the multilayer perceptron, backpropagation (with dropout and L2), the three optimizers and the training loop with early stopping. The experiment scripts only call these functions with the settings stated in each question.

```
import numpy as np, time

def sigmoid(z):
    return 1.0/(1.0+np.exp(-np.clip(z,-60,60)))

def act_f(name, z):
    if name=='relu': return np.maximum(0,z)
    if name=='tanh': return np.tanh(z)
    if name=='sigmoid': return sigmoid(z)

def act_d(name, z):
    if name=='relu': return (z>0).astype(float)
    if name=='tanh': return 1-np.tanh(z)**2
    if name=='sigmoid':
        s=sigmoid(z); return s*(1-s)

def bce(p,y,eps=1e-12):
    p=np.clip(p,eps,1-eps); y=y.reshape(-1,1) if p.ndim==2 else y
    return float(-np.mean(y*np.log(p)+(1-y)*np.log(1-p)))

class MLP:
    def __init__(self, sizes, act='relu', seed=0, dropout=0.0, l2=0.0):
        rng=np.random.default_rng(seed)
        self.sizes=list(sizes); self.act=act; self.dropout=dropout; self.l2=l2
        self.W=[]; self.b=[]
        for i in range(len(sizes)-1):
            s=np.sqrt(2.0/sizes[i]) if act=='relu' else np.sqrt(1.0/sizes[i])
            self.W.append(rng.normal(0,s,(sizes[i],sizes[i+1])))
            self.b.append(np.zeros(sizes[i+1]))
        self.rng=np.random.default_rng(seed+1000)
    def n_params(self):
        return sum(w.size for w in self.W)+sum(b.size for b in self.b)
    def forward(self,X,train=False):
        A=[X]; Z=[]; M=[]; a=X; L=len(self.W)
        for i in range(L):
            z=a@self.W[i]+self.b[i]; Z.append(z)
            if i<L-1:
                a=act_f(self.act,z)
                if train and self.dropout>0:
                    m=(self.rng.random(a.shape)>=self.dropout)/(1-self.dropout)
                    a=a*m; M.append(m)
                else: M.append(None)
            else:
                a=sigmoid(z); M.append(None)
            A.append(a)
        return A,Z,M
    def predict_proba(self,X):
        return self.forward(X)[0][-1].ravel()
    def evaluate(self,X,y):
        p=self.predict_proba(X)
        return bce(p,y), float(np.mean((p>=0.5)==y))
    def grads(self,X,y):
        A,Z,M=self.forward(X,train=True)
        n=len(y); L=len(self.W)
        dz=(A[-1]-y.reshape(-1,1))/n
        gW=[None]*L; gb=[None]*L
        for i in reversed(range(L)):
            gW[i]=A[i].T@dz+self.l2*self.W[i]
            gb[i]=dz.sum(0)
            if i>0:
                da=dz@self.W[i].T
                if M[i-1] is not None: da=da*M[i-1]
                dz=da*act_d(self.act,Z[i-1])
        return gW,gb
```

```
    def get(self): return [w.copy() for w in self.W],[b.copy() for b in self.b]
    def set(self,s): self.W=[w.copy() for w in s[0]]; self.b=[b.copy() for b in s[1]]
    def wnorm(self): return float(np.sqrt(sum((w**2).sum() for w in self.W)))

class Opt:
    def __init__(self,name,lr,params,beta=0.9,b1=0.9,b2=0.999,eps=1e-8):
        self.name=name; self.lr=lr; self.beta=beta; self.b1=b1; self.b2=b2;
self.eps=eps; self.t=0
        self.m=[np.zeros_like(p) for p in params]; self.v=[np.zeros_like(p) for p in
params]
    def step(self,params,grads):
        self.t+=1
        for i,(p,g) in enumerate(zip(params,grads)):
            if self.name=='sgd': p-=self.lr*g
            elif self.name=='momentum':
                self.m[i]=self.beta*self.m[i]+g; p-=self.lr*self.m[i]
            elif self.name=='adam':
                self.m[i]=self.b1*self.m[i]+(1-self.b1)*g
                self.v[i]=self.b2*self.v[i]+(1-self.b2)*g*g
                mh=self.m[i]/(1-self.b1**self.t); vh=self.v[i]/(1-self.b2**self.t)
                p-=self.lr*mh/(np.sqrt(vh)+self.eps)

def train(model,Xtr,ytr,Xva,yva,opt='adam',lr=0.01,bs=32,epochs=100,patience=None,
          min_delta=1e-4,restore_best=True,seed=0):
    params=model.W+model.b
    o=Opt(opt,lr,params)
    rng=np.random.default_rng(seed+500)
    H={k:[] for k in ['tr_loss','tr_acc','va_loss','va_acc']}
    best=np.inf; best_ep=0; snap=None; wait=0
    t0=time.perf_counter(); n=len(ytr); L=len(model.W); diverged=False
    for ep in range(epochs):
        idx=rng.permutation(n)
        for s in range(0,n,bs):
            b=idx[s:s+bs]
            gW,gb=model.grads(Xtr[b],ytr[b])
            o.step(model.W+model.b,gW+gb)
        tl,ta=model.evaluate(Xtr,ytr); vl,va=model.evaluate(Xva,yva)
        if not np.isfinite(tl) or not np.isfinite(vl): diverged=True
        for k,v in zip(H.keys(),[tl,ta,vl,va]): H[k].append(v)
        if vl<best-min_delta:
            best=vl; best_ep=ep; snap=model.get(); wait=0
        else: wait+=1
        if patience and wait>=patience: break
    T=time.perf_counter()-t0
    if restore_best and snap is not None: model.set(snap)
    return
dict(H=H,best_ep=best_ep,time=T,epochs_run=len(H['tr_loss']),diverged=diverged)
```